

# Blockchain-Based Supply Chain Management System

## Complete Architecture & Implementation Guide

### Table of Contents

- [Project Overview](#)
- [System Architecture](#)
- [Technology Stack](#)
- [Project Structure](#)
- [Implementation Steps](#)
- [Detailed Component Breakdown](#)

## 1. Project Overview

### System Description

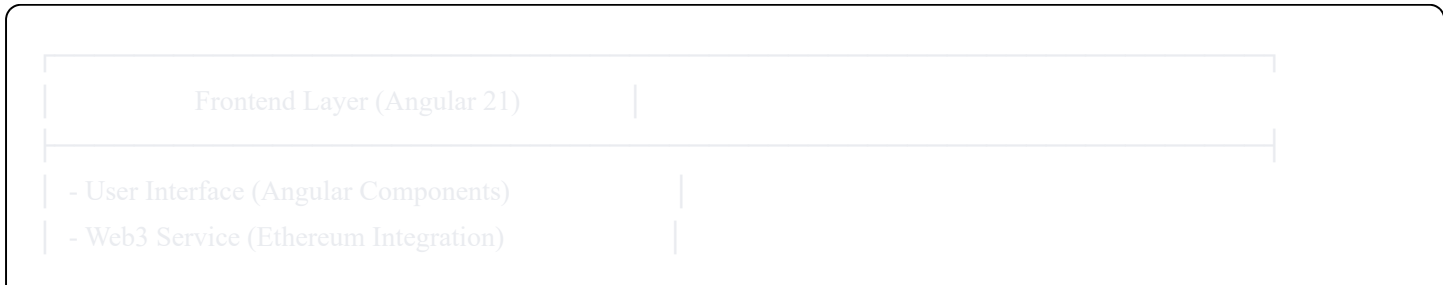
A decentralized supply chain management system that tracks products from order creation through the entire supply chain lifecycle: Order → Raw Material Supplier → Manufacturer → Distributor → Retailer → Consumer.

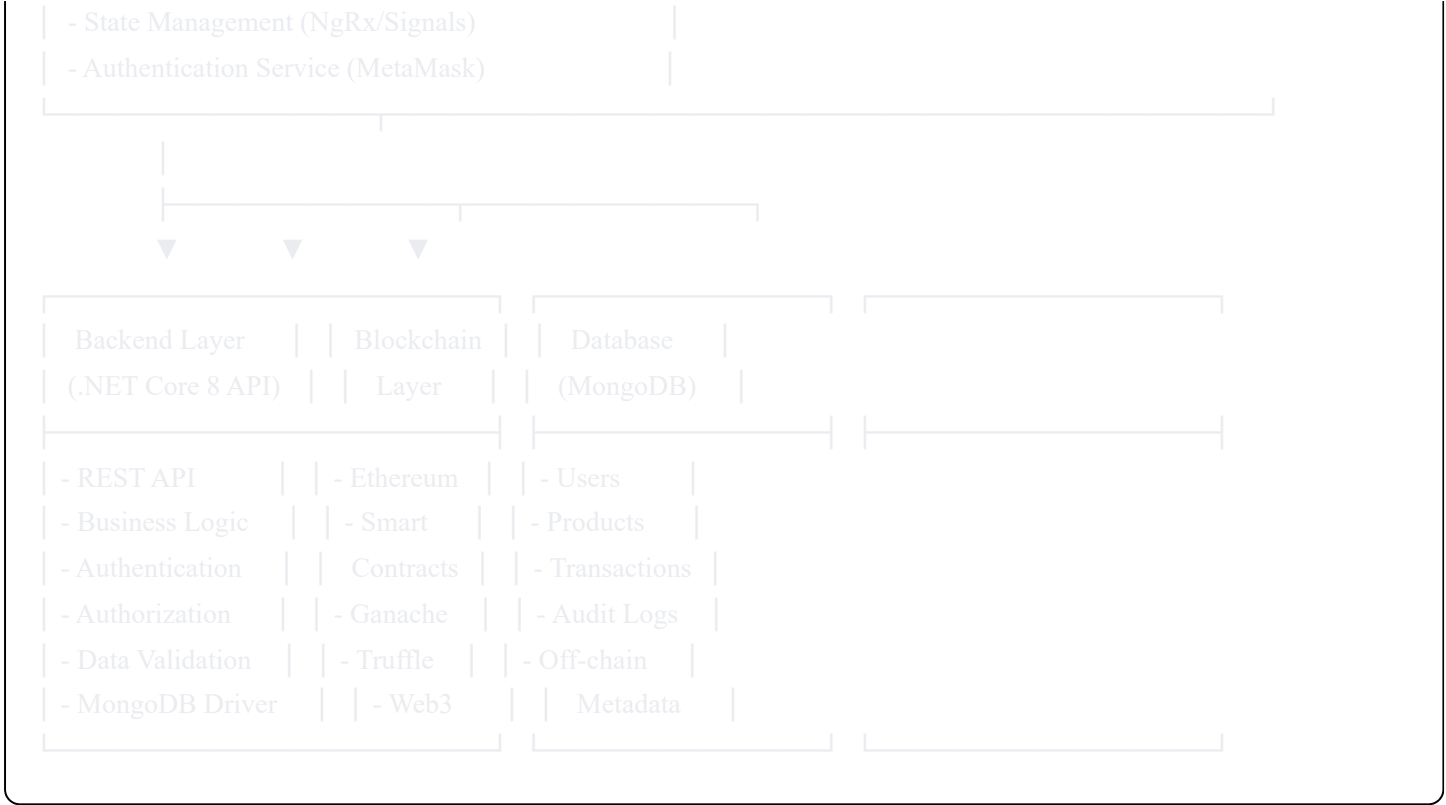
### Key Features

- Blockchain Integration:** Immutable record of all transactions using Ethereum
- Smart Contracts:** Automated verification and state transitions
- Role-Based Access:** Different permissions for each supply chain participant
- Real-time Tracking:** Monitor product journey at each stage
- Transparency:** All stakeholders can verify product authenticity
- Security:** Cryptographic verification using MetaMask

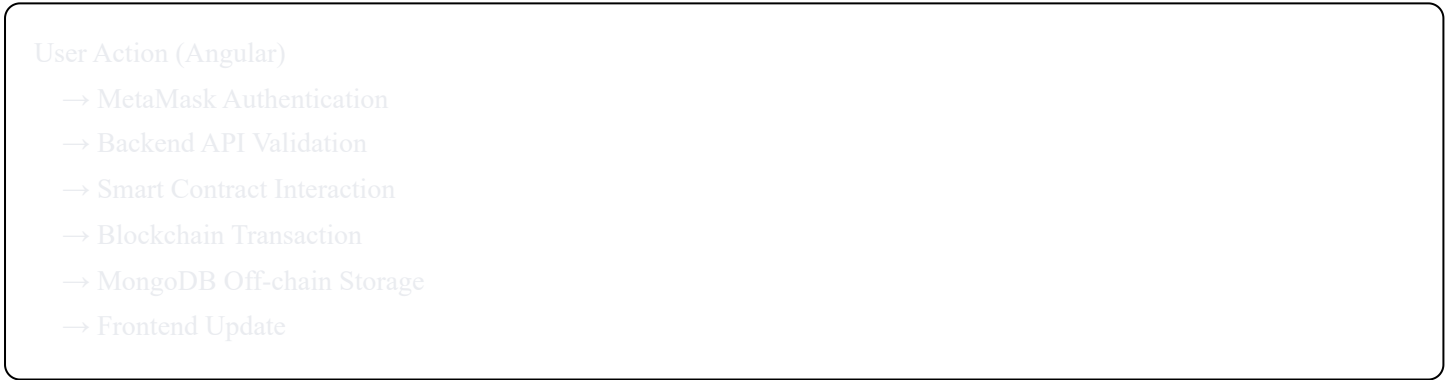
## 2. System Architecture

### High-Level Architecture





### Data Flow Architecture



## 3. Technology Stack

### Frontend

- **Angular 21:** Latest version with standalone components and signals
- **TypeScript:** Type-safe development
- **Web3.js/Ethers.js:** Ethereum blockchain interaction
- **RxJS:** Reactive programming
- **Tailwind CSS:** Modern styling
- **NgRx/Signals:** State management

### Backend

- **.NET Core 8:** Web API

- **Entity Framework Core:** ORM (optional with MongoDB)
- **MongoDB.Driver:** Database operations
- **JWT Authentication:** Secure API access
- **Nethereum:** .NET Ethereum integration library

## Blockchain

- **Ethereum:** Blockchain platform
- **Solidity:** Smart contract language
- **Ganache:** Local Ethereum blockchain
- **Truffle:** Smart contract development framework
- **MetaMask:** Wallet and authentication
- **Web3.js:** JavaScript Ethereum API

## Database

- **MongoDB Atlas:** Cloud-hosted NoSQL database
  - **Collections:** Users, Products, Orders, Transactions, AuditLogs
- 

## 4. Project Structure

```
supply-chain-blockchain/
|
|— blockchain/                # Smart Contracts
|   |— contracts/
|   |   |— SupplyChain.sol      # Main supply chain contract
|   |   |— AccessControl.sol    # Role-based access control
|   |   |— ProductRegistry.sol  # Product management
|   |   |— Migrations.sol       # Truffle migrations
|   |— migrations/
|   |   |— 2_deploy_contracts.js
|   |— test/
|   |   |— supplychain.test.js
|   |— truffle-config.js
|
|— backend/                  # .NET Core API
|   |— SupplyChain.API/
|   |   |— Controllers/
|   |   |   |— AuthController.cs
|   |   |   |— ProductController.cs
|   |   |   |— OrderController.cs
|   |   |   |— BlockchainController.cs
```

```
| | | └── UserController.cs
| | | └── Services/
| | |   ├── IBlockchainService.cs
| | |   ├── BlockchainService.cs
| | |   ├── IProductService.cs
| | |   ├── ProductService.cs
| | |   └── AuthService.cs
| | | └── Models/
| | |   ├── Entities/
| | |     ├── User.cs
| | |     ├── Product.cs
| | |     ├── Order.cs
| | |     └── Transaction.cs
| | |   └── DTOs/
| | |     ├── ProductDto.cs
| | |     ├── OrderDto.cs
| | |     └── UserDto.cs
| | | └── Data/
| | |   ├── MongoDBContext.cs
| | |   └── IMongoDbContext.cs
| | | └── Middleware/
| | |   ├── ExceptionMiddleware.cs
| | |   └── JwtMiddleware.cs
| | | └── Configuration/
| | |   ├── MongoDBSettings.cs
| | |   └── BlockchainSettings.cs
| | └── Program.cs
| └── SupplyChain.API.sln
└── frontend/          # Angular Application
    ├── src/
    |   ├── app/
    |   |   ├── core/
    |   |   |   ├── services/
    |   |   |   |   ├── web3.service.ts
    |   |   |   |   ├── blockchain.service.ts
    |   |   |   |   ├── auth.service.ts
    |   |   |   |   └── api.service.ts
    |   |   |   ├── guards/
    |   |   |   |   ├── auth.guard.ts
    |   |   |   |   └── role.guard.ts
    |   |   |   ├── interceptors/
    |   |   |   |   └── jwt.interceptor.ts
    |   |   |   └── models/
    |   |   |       ├── product.model.ts
    |   |   |       ├── order.model.ts
    |   |   |       └── user.model.ts
```

```
| | | — features/
| | | — dashboard/
| | | — products/
| | | — orders/
| | | — tracking/
| | | — admin/
| | | — shared/
| | | — components/
| | | — directives/
| | | — pipes/
| | — app.component.ts
| — assets/
| — environments/
— angular.json
```

---

## 5. Implementation Steps

### Phase 1: Environment Setup (Week 1)

#### Step 1.1: Install Required Software

```
bash

# Node.js and npm
# Download from nodejs.org (v18+ recommended)

# .NET 8 SDK
# Download from dotnet.microsoft.com

# MongoDB
# Sign up at mongodb.com/atlas for cloud database

# Ganache
# Download from trufflesuite.com/ganache

# Truffle
npm install -g truffle

# Angular CLI
npm install -g @angular/cli@21

# MetaMask
# Install browser extension from metamask.io
```

#### Step 1.2: Configure Ganache

1. Open Ganache
2. Create new workspace
3. Configure:
  - Port: 7545
  - Network ID: 5777
  - Gas Price: 200000000000
  - Gas Limit: 6721975
4. Note down account addresses and private keys

### Step 1.3: Setup MongoDB Atlas

1. Create account at [mongodb.com](https://mongodb.com)
  2. Create new cluster (Free tier M0)
  3. Create database user
  4. Whitelist IP (0.0.0.0/0 for development)
  5. Get connection string
  6. Create databases:
    - SupplyChainDB
    - Collections: Users, Products, Orders, Transactions, AuditLogs
- 

## Phase 2: Smart Contract Development (Week 1-2)

### Step 2.1: Initialize Truffle Project

```
bash
mkdir blockchain
cd blockchain
truffle init
```

### Step 2.2: Create Smart Contracts

**File: contracts/SupplyChain.sol**

```
solidity
```

```
// SPDX-License-Identifier: MIT
```

```
pragma solidity ^0.8.19;
```

```
contract SupplyChain {
```

```
    // Enums
```

```
    enum Role { None, Owner, RawMaterialSupplier, Manufacturer, Distributor, Retailer, Consumer }
```

```
    enum Stage { Ordered, RawMaterialSupplied, Manufactured, Distributed, Retailed, Sold }
```

```
    // Structs
```

```
    struct Product {
```

```
        uint256 id;
```

```
        string name;
```

```
        string description;
```

```
        uint256 price;
```

```
        address currentOwner;
```

```
        Stage currentStage;
```

```
        uint256 timestamp;
```

```
        bool exists;
```

```
    }
```

```
    struct User {
```

```
        address userAddress;
```

```
        string name;
```

```
        string email;
```

```
        Role role;
```

```
        bool isActive;
```

```
        uint256 registrationDate;
```

```
    }
```

```
    struct StageHistory {
```

```
        Stage stage;
```

```
        address handler;
```

```
        uint256 timestamp;
```

```
        string location;
```

```
        string remarks;
```

```
    }
```

```
    // State Variables
```

```
    address public contractOwner;
```

```
    uint256 public productCounter;
```

```
    // Mappings
```

```
    mapping(uint256 => Product) public products;
```

```
    mapping(address => User) public users;
```

```
    mapping(uint256 => StageHistory[]) public productHistory;
```

```
    mapping(address => bool) public registeredUsers;
```

```

// Events
event UserRegistered(address indexed userAddress, Role role, uint256 timestamp);
event ProductCreated(uint256 indexed productId, string name, address indexed owner);
event StageUpdated(uint256 indexed productId, Stage newStage, address indexed handler);
event OwnershipTransferred(uint256 indexed productId, address indexed from, address indexed to);


// Modifiers
modifier onlyOwner() {
    require(msg.sender == contractOwner, "Only contract owner can call this");
    _;
}

modifier onlyRegistered() {
    require(registeredUsers[msg.sender], "User not registered");
    _;
}

modifier hasRole(Role _role) {
    require(users[msg.sender].role == _role, "Unauthorized role");
    _;
}

modifier productExists(uint256 _productId) {
    require(products[_productId].exists, "Product does not exist");
    _;
}

modifier validStageTransition(uint256 _productId, Stage _newStage) {
    Stage currentStage = products[_productId].currentStage;
    require(uint8(_newStage) == uint8(currentStage) + 1, "Invalid stage transition");
    _;
}


// Constructor
constructor() {
    contractOwner = msg.sender;
    users[msg.sender] = User({
        userAddress: msg.sender,
        name: "Contract Owner",
        email: "owner@supplychain.com",
        role: Role.Owner,
        isActive: true,
        registrationDate: block.timestamp
    });
    registeredUsers[msg.sender] = true;
}

```

```
// User Management Functions
```

```
function registerUser(  
  address _userAddress,  
  string memory _name,  
  string memory _email,  
  Role _role  
) public onlyOwner {  
  require(!registeredUsers[_userAddress], "User already registered");  
  require(_role != Role.None && _role != Role.Owner, "Invalid role");  
  
  users[_userAddress] = User({  
    userAddress: _userAddress,  
    name: _name,  
    email: _email,  
    role: _role,  
    isActive: true,  
    registrationDate: block.timestamp  
  });  
  
  registeredUsers[_userAddress] = true;  
  emit UserRegistered(_userAddress, _role, block.timestamp);  
}
```

```
function updateUserStatus(address _userAddress, bool _isActive) public onlyOwner {  
  require(registeredUsers[_userAddress], "User not found");  
  users[_userAddress].isActive = _isActive;  
}
```

```
// Product Management Functions
```

```
function createProduct(  
  string memory _name,  
  string memory _description,  
  uint256 _price  
) public onlyRegistered returns (uint256) {  
  productCounter++;  
  
  products[productCounter] = Product({  
    id: productCounter,  
    name: _name,  
    description: _description,  
    price: _price,  
    currentOwner: msg.sender,  
    currentStage: Stage.Ordered,  
    timestamp: block.timestamp,  
    exists: true  
  });  
}
```

```

// Add initial stage to history
productHistory[productCounter].push(StageHistory({
    stage: Stage.Ordered,
    handler: msg.sender,
    timestamp: block.timestamp,
    location: "Origin",
    remarks: "Product ordered"
}));

emit ProductCreated(productCounter, _name, msg.sender);
return productCounter;
}

function updateProductStage(
    uint256 _productId,
    Stage _newStage,
    string memory _location,
    string memory _remarks
) public
    onlyRegistered
    productExists(_productId)
    validStageTransition(_productId, _newStage)
{
    Product storage product = products[_productId];

    // Verify role matches stage
    require(verifyRoleForStage(users[msg.sender].role, _newStage), "Invalid role for this stage");

    product.currentStage = _newStage;
    product.currentOwner = msg.sender;
    product.timestamp = block.timestamp;

    // Add to history
    productHistory[_productId].push(StageHistory({
        stage: _newStage,
        handler: msg.sender,
        timestamp: block.timestamp,
        location: _location,
        remarks: _remarks
    }));

    emit StageUpdated(_productId, _newStage, msg.sender);
    emit OwnershipTransferred(_productId, product.currentOwner, msg.sender);
}

function verifyRoleForStage(Role _role, Stage _stage) private pure returns (bool) {

```

```

    if (_stage == Stage.Ordered) return true;
    if (_stage == Stage.RawMaterialSupplied) return _role == Role.RawMaterialSupplier;
    if (_stage == Stage.Manufactured) return _role == Role.Manufacturer;
    if (_stage == Stage.Distributed) return _role == Role.Distributor;
    if (_stage == Stage.Retailed) return _role == Role.Retailer;
    if (_stage == Stage.Sold) return _role == Role.Consumer;
    return false;
}

// Query Functions
function getProduct(uint256 _productId) public view returns (Product memory) {
    return products[_productId];
}

function getProductHistory(uint256 _productId) public view returns (StageHistory[] memory) {
    return productHistory[_productId];
}

function getUser(address _userAddress) public view returns (User memory) {
    require(registeredUsers[_userAddress], "User not found");
    return users[_userAddress];
}

function getUserRole(address _userAddress) public view returns (Role) {
    return users[_userAddress].role;
}

function isUserRegistered(address _userAddress) public view returns (bool) {
    return registeredUsers[_userAddress];
}

function getTotalProducts() public view returns (uint256) {
    return productCounter;
}
}

```

## Step 2.3: Configure Truffle

### File: truffle-config.js

```
javascript
```

```
module.exports = {
  networks: {
    development: {
      host: "127.0.0.1",
      port: 7545,
      network_id: "5777",
      gas: 6721975,
      gasPrice: 20000000000
    }
  },

  compilers: {
    solc: {
      version: "0.8.19",
      settings: {
        optimizer: {
          enabled: true,
          runs: 200
        }
      }
    }
  }
};
```

## Step 2.4: Create Migration

**File: migrations/2\_deploy\_contracts.js**

```
javascript

const SupplyChain = artifacts.require("SupplyChain");

module.exports = function(deployer) {
  deployer.deploy(SupplyChain);
};
```

## Step 2.5: Compile and Deploy

```
bash
```

```
# Compile contracts
```

```
truffle compile
```

```
# Deploy to Ganache
```

```
truffle migrate --reset
```

```
# Test contracts
```

```
truffle test
```

---

## Phase 3: Backend Development (.NET Core 8) (Week 2-3)

### Step 3.1: Create .NET Project

```
bash
```

```
mkdir backend
```

```
cd backend
```

```
dotnet new webapi -n SupplyChain.API
```

```
cd SupplyChain.API
```

### Step 3.2: Install NuGet Packages

```
bash
```

```
dotnet add package MongoDB.Driver
```

```
dotnet add package Nethereum.Web3
```

```
dotnet add package Microsoft.AspNetCore.Authentication.JwtBearer
```

```
dotnet add package System.IdentityModel.Tokens.Jwt
```

```
dotnet add package BCrypt.Net-Next
```

```
dotnet add package AutoMapper.Extensions.Microsoft.DependencyInjection
```

### Step 3.3: Configuration Files

#### File: appsettings.json

```
json
```

```

{
  "MongoDbSettings": {
    "ConnectionString": "mongodb+srv://username:password@cluster.mongodb.net/",
    "DatabaseName": "SupplyChainDB",
    "UsersCollectionName": "Users",
    "ProductsCollectionName": "Products",
    "OrdersCollectionName": "Orders",
    "TransactionsCollectionName": "Transactions",
    "AuditLogsCollectionName": "AuditLogs"
  },
  "BlockchainSettings": {
    "NodeUrl": "http://127.0.0.1:7545",
    "ContractAddress": "YOUR_DEPLOYED_CONTRACT_ADDRESS",
    "ChainId": 5777,
    "GasPrice": 20000000000,
    "GasLimit": 6721975
  },
  "JwtSettings": {
    "Secret": "YourSuperSecretKeyForJWTTokenGeneration123456",
    "Issuer": "SupplyChainAPI",
    "Audience": "SupplyChainApp",
    "ExpirationInMinutes": 1440
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*"
}

```

### Step 3.4: Models and Entities

#### File: Models/Entities/User.cs

```
csharp
```

```
using MongoDB.Bson;
using MongoDB.Bson.Serialization.Attributes;

namespace SupplyChain.API.Models.Entities
{
    public class User
    {
        [BsonId]
        [BsonRepresentation(BsonType.ObjectId)]
        public string Id { get; set; }

        [BsonElement("walletAddress")]
        public string WalletAddress { get; set; }

        [BsonElement("name")]
        public string Name { get; set; }

        [BsonElement("email")]
        public string Email { get; set; }

        [BsonElement("passwordHash")]
        public string PasswordHash { get; set; }

        [BsonElement("role")]
        public UserRole Role { get; set; }

        [BsonElement("phoneNumber")]
        public string PhoneNumber { get; set; }

        [BsonElement("company")]
        public string Company { get; set; }

        [BsonElement("address")]
        public string Address { get; set; }

        [BsonElement("isActive")]
        public bool IsActive { get; set; }

        [BsonElement("isVerified")]
        public bool IsVerified { get; set; }

        [BsonElement("createdAt")]
        public DateTime CreatedAt { get; set; }

        [BsonElement("updatedAt")]
        public DateTime UpdatedAt { get; set; }
    }
}
```

```
}

public enum UserRole
{
    None = 0,
    Owner = 1,
    RawMaterialSupplier = 2,
    Manufacturer = 3,
    Distributor = 4,
    Retailer = 5,
    Consumer = 6
}
}
```

### File: Models/Entities/Product.cs

```
csharp
```

```
using MongoDB.Bson;
using MongoDB.Bson.Serialization.Attributes;

namespace SupplyChain.API.Models.Entities
{
    public class Product
    {
        [BsonId]
        [BsonRepresentation(BsonType.ObjectId)]
        public string Id { get; set; }

        [BsonElement("blockchainId")]
        public long BlockchainId { get; set; }

        [BsonElement("name")]
        public string Name { get; set; }

        [BsonElement("description")]
        public string Description { get; set; }

        [BsonElement("category")]
        public string Category { get; set; }

        [BsonElement("price")]
        public decimal Price { get; set; }

        [BsonElement("quantity")]
        public int Quantity { get; set; }

        [BsonElement("sku")]
        public string SKU { get; set; }

        [BsonElement("barcode")]
        public string Barcode { get; set; }

        [BsonElement("currentStage")]
        public ProductStage CurrentStage { get; set; }

        [BsonElement("currentOwnerAddress")]
        public string CurrentOwnerAddress { get; set; }

        [BsonElement("imageUrl")]
        public string ImageUrl { get; set; }

        [BsonElement("specifications")]
        public Dictionary<string, string> Specifications { get; set; }
    }
}
```

```
[BsonElement("createdBy")]
public string CreatedBy { get; set; }

[BsonElement("createdAt")]
public DateTime CreatedAt { get; set; }

[BsonElement("updatedAt")]
public DateTime UpdatedAt { get; set; }
}

public enum ProductStage
{
    Ordered = 0,
    RawMaterialSupplied = 1,
    Manufactured = 2,
    Distributed = 3,
    Retailed = 4,
    Sold = 5
}
}
```

## File: Models/Entities/Order.cs

```
csharp
```

```
using MongoDB.Bson;
using MongoDB.Bson.Serialization.Attributes;

namespace SupplyChain.API.Models.Entities
{
    public class Order
    {
        [BsonId]
        [BsonRepresentation(BsonType.ObjectId)]
        public string Id { get; set; }

        [BsonElement("orderNumber")]
        public string OrderNumber { get; set; }

        [BsonElement("productId")]
        public string ProductId { get; set; }

        [BsonElement("customerId")]
        public string CustomerId { get; set; }

        [BsonElement("quantity")]
        public int Quantity { get; set; }

        [BsonElement("totalAmount")]
        public decimal TotalAmount { get; set; }

        [BsonElement("status")]
        public OrderStatus Status { get; set; }

        [BsonElement("shippingAddress")]
        public Address ShippingAddress { get; set; }

        [BsonElement("expectedDeliveryDate")]
        public DateTime? ExpectedDeliveryDate { get; set; }

        [BsonElement("actualDeliveryDate")]
        public DateTime? ActualDeliveryDate { get; set; }

        [BsonElement("createdAt")]
        public DateTime CreatedAt { get; set; }

        [BsonElement("updatedAt")]
        public DateTime UpdatedAt { get; set; }
    }

    public class Address
```

```
{  
    public string Street { get; set; }  
    public string City { get; set; }  
    public string State { get; set; }  
    public string Country { get; set; }  
    public string PostalCode { get; set; }  
}  
  
public enum OrderStatus  
{  
    Pending,  
    Processing,  
    InTransit,  
    Delivered,  
    Cancelled  
}  
}
```

#### File: Models/Entities/Transaction.cs

```
csharp
```

```

using MongoDB.Bson;
using MongoDB.Bson.Serialization.Attributes;

namespace SupplyChain.API.Models.Entities
{
    public class Transaction
    {
        [BsonId]
        [BsonRepresentation(BsonType.ObjectId)]
        public string Id { get; set; }

        [BsonElement("transactionHash")]
        public string TransactionHash { get; set; }

        [BsonElement("blockNumber")]
        public long BlockNumber { get; set; }

        [BsonElement("productId")]
        public long ProductId { get; set; }

        [BsonElement("fromAddress")]
        public string FromAddress { get; set; }

        [BsonElement("toAddress")]
        public string ToAddress { get; set; }

        [BsonElement("stage")]
        public ProductStage Stage { get; set; }

        [BsonElement("location")]
        public string Location { get; set; }

        [BsonElement("remarks")]
        public string Remarks { get; set; }

        [BsonElement("gasUsed")]
        public long GasUsed { get; set; }

        [BsonElement("timestamp")]
        public DateTime Timestamp { get; set; }
    }
}

```

This is Part 1 of the comprehensive guide. The document continues with:

- MongoDB Context setup

- Services implementation
- Controllers
- Frontend Angular setup
- Complete integration
- Testing strategies
- Deployment guide

Would you like me to continue with the next parts?